



S.G. Ganesh

Understanding the Lifetime of a C Program

How does the OS start a C program? What happens when a C program terminates? In this month's column, we'll cover these questions from the perspective of someone who is a beginner in C.

A compiled C program is a native executable, and is like any other executable to the OS—the OS doesn't even know that it's a C program. When we call the executable program, the OS does a lot of work in the background. First, the loader allocates space for the process, and loads the program into memory. Then other activities are performed; for example, the necessary data initialisations (remember—local static and global variables are initialised before *main()* starts).

The OS then calls a 'start-up routine'—say *_init()* or *_main()*—to perform tasks like setting up environmental variables. Then the start-up routine calls the *main()* function, which is logically (for us) the starting point for C program execution.

The *main()* function is special in C, and has many differences from ordinary functions. Some examples are: the parameters to *main()* are passed from the command-line; we can define *main()* with either zero or two arguments; the return type of *main()* is always *int* (some compilers use *void*, but that is non-standard).

Traditionally, the name of the first parameter to *main()* is *argc*, and it has the number of command-line arguments passed; the name stands for 'argument count'. The second argument is typically named *argv*, and it has the actual list of arguments; the name stands for 'argument vector' ('vector' means 'array' in systems programming). The value *argv[0]* is the name used to call the C program. Some compilers allow *envp* as a third argument, which has the list of environmental variables (this is also a non-standard extension).

The *main()* function, in turn, calls any functions that are given in its body. Such function calls are the actual functionality that the program provides. In the program, we can register our own 'clean-up' functions that should be called before the termination of the program; we register such functions by calling the *atexit()* function.

For each function call made in the program, a 'stack frame' is created. Each 'stack frame' has details like the actual incoming parameters, information about the old stack pointer, space for local variables, temporary storage area, outgoing

parameters (which become incoming parameters for the called function), etc. As the program executes, stack frames are created and destroyed as required. If nothing abnormal happens, all stack frames will be gone and the control finally returns to *main()*; now only the stack frame of *main()* will be left. Then *main()* returns to the start-up routine, which calls *exit()* to terminate the program with the return value given by *main()*.

There are two ways to terminate a program: normal termination and abnormal termination.

A program will terminate abnormally:

- if we explicitly call *abort()* (which in turn raises *SIGABRT*);
- if a runtime error happens (say, divide-by-zero);
- if we raise a signal (say *SIGTERM*) without a handler.

In these cases, *exit()* will not be called, and thus, the exit handlers we registered using *atexit()* will not be called; file streams will not be flushed properly, and the program will 'dump core'; also, a non-zero integer value is returned to the OS.

The program will terminate normally if the program calls *return* explicitly from *main()*; if the control reaches the last statement of *main()*; or if *exit()* is called from anywhere in the program (yes, calling *exit()* explicitly is normal termination!). The integer value 0 is returned to the OS.

In normal program termination, the *exit()* function calls all the exit handlers registered using the *atexit()* function. All files and streams are flushed, and the control returns to the OS. This is the right way to terminate a program. The OS clears the resources (like file handlers), and removes the process from memory, and that's how the life of a C program comes to an end. 

By: S.G. Ganesh

The author works in Siemens (Corporate Technology), Bangalore. His latest book is the best-seller *Cracking the C, C++ and Java Interview*, published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.